SWENG 894 Capstone

# MicroMatch

## Software Requirements Specification

**Chastidy Joanem**

M.S. Software Engineering | April 2026

MicroMatch Repository Link

# Table of Contents

# 1. Introduction

## 1.1 Mission Statement / Concept of Operations

MicroMatch was created to solve a critical gap in early-career professional development: university students possess theoretical skills but lack practical, verifiable project experience, while organizations need short-term skilled contributors but lack a structured, trusted channel to find and engage them.

The mission of MicroMatch is to democratize access to professional project experience by providing a governed, intelligent micro-internship marketplace that reduces friction for both parties. Students gain real-world credentials and feedback-backed credibility. Organizations gain access to a motivated, skill-filtered talent pool without the overhead of a formal internship program.

The Concept of Operations (CONOPS) for MicroMatch describes a full lifecycle marketplace where:

- Organizations post project listings with required skills, description, and duration.
- Students discover projects through keyword search and a personalized smart recommendation feed ranked by a weighted matching algorithm.
- Students apply, and organizations review applicants, accepting or rejecting based on profile fit.
- Accepted students complete project work and submit deliverables through the platform.
- Organizations review deliverables and mark projects complete when satisfied.
- Both parties submit post-project feedback ratings, which feed into the student's credibility profile.
- The platform tracks all activity and automatically awards achievement badges to students based on verified milestones.

The system is governed by Role-Based Access Control (RBAC) enforced at every API endpoint, ensuring that students, organizations, and administrators can only perform actions appropriate to their role.

## 1.2 Scope of the System

MicroMatch is a web-based, full-stack application. The scope of the delivered MVP encompasses the following capabilities:

| Capability Area | In Scope | Out of Scope |
|---|---|---|
| User Accounts | Registration, login, JWT authentication, three roles (student, organization, admin) | OAuth2 social login, account deletion self-service |
| Profiles | Student profile with skills, bio, portfolio links, auto-awarded badges; organization profile | Resume upload, file attachments, profile photo uploads |
| Project Listings | Create, browse, search, and view project details | Paid/sponsored listings, project templates |
| Applications | Submit, track, accept/reject with notifications | Multi-stage application interviews, video submissions |
| Deliverables | Text/link submission, org review, approve/request revision | File hosting, automated plagiarism checking |

| Capability Area | In Scope | Out of Scope |
|---|---|---|
| Feedback | 1-5 star rating + comment from both parties | Weighted dispute resolution, public reviews |
| Messaging | Project-scoped direct messaging, auto-polling | Group chats, multimedia messaging, file sharing |
| Recommendations | Weighted 5-factor matching algorithm, ranked feed | Collaborative filtering, ML-based personalization |
| Analytics | Role-specific stat cards and bar chart on home dashboard | Custom date-range reports, CSV exports |
| Notifications | Event-driven bell notifications for all lifecycle events | Push notifications, email/SMS delivery |
| Administration | System request logs, content moderation tools | Automated content flagging, user banning workflows |

## 1.3 Target Audience

MicroMatch serves three distinct user groups:

| User Role | Description | Primary Goals |
|---|---|---|
| Student | University-enrolled students seeking practical project experience | Discover skill-matched projects, build a verified credential portfolio, earn achievement badges, communicate with organizations |
| Organization | Companies or research groups with short-term project needs | Post project listings, find qualified students, review deliverables, provide feedback, manage active project communications |
| Administrator | Platform operators responsible for system health and compliance | Monitor system logs for anomalies, moderate content, ensure platform integrity |

## 1.4 Document Overview

This Software Requirements Specification (SRS) is organized as follows:

- **Section 1 — Introduction:** Mission statement, system scope, target audience, and document overview.
- **Section 2 — System Overview:** System context, feature list with Use Cases/User Stories, user-action mapping, constraints, and assumptions.
- **Section 3 — Functional Requirements:** Complete enumeration of all 19 functional requirements (FR-1 through FR-19) mapped to User Stories, with full acceptance criteria.
- **Section 4 — Non-Functional Requirements:** Eight NFRs covering security, correctness, testability, performance, maintainability, observability, deployability, and usability, with business rationale for each.

- **Section 5 — Algorithmic Component:** Detailed specification of the Smart Matching & Ranking Algorithm (SA-1), including problem statement, logic, scoring formula, and mapping to requirements.

# 2. System Overview

## 2.1 System Context

MicroMatch operates as a three-tier web application. The React SPA frontend communicates exclusively with the FastAPI REST backend via HTTP/JSON. The backend enforces all business logic, authentication, and authorization before persisting data to a PostgreSQL database. No client has direct database access.

| Tier | Technology | Responsibility |
|---|---|---|
| Presentation | React + Tailwind CSS | Renders UI, manages routing, issues authenticated API calls |
| Application | FastAPI (Python 3.11) + Pydantic V2 | REST API, JWT auth, RBAC, business logic, event triggers |
| Persistence | PostgreSQL 15 (Docker) | Relational data storage, FK constraints, ACID compliance |

A cross-cutting logging middleware intercepts every HTTP request and persists a SystemLog record, enabling full observability of all platform activity.

## 2.2 Significant Features and User Stories

| Feature | User Story ID | User Story | Role |
|---|---|---|---|
| Account Management | US-1 | As a user, I want to register and log in so that I can access the platform. | All |
| Student Profile | US-2 | As a student, I want to create and edit my profile with skills and bio so that organizations can evaluate my fit. | Student |
| Organization Profile | US-3 | As an organization, I want to create a profile so that students can understand who we are. | Organization |
| Project Posting | US-4 | As an organization, I want to post a project listing with required skills and duration so that students can apply. | Organization |
| Project Discovery | US-5 | As a student, I want to browse and search open projects so that I can find relevant opportunities. | Student |
| Apply to Projects | US-6 | As a student, I want to apply to a project so that the organization can review my application. | Student |
| Review Applications | US-7 | As an organization, I want to review student applications and accept or reject them so that I can choose the best fit. | Organization |
| Track Status | US-8 | As a student, I want to track the status of my applications so that I know where I stand. | Student |
| Accept/Reject | US-9 | As an organization, I want to accept or reject applicants so that I can manage my project team. | Organization |
| Submit Deliverables | US-10 | As an accepted student, I want to submit my project deliverable so that the organization can review my work. | Student |

| Feature | User Story ID | User Story | Role |
|---|---|---|---|
| Review Deliverables | US-11 | As an organization, I want to review submitted deliverables and approve or request revisions. | Organization |
| Mark Complete | US-12 | As an organization, I want to mark a project complete so that the feedback cycle can begin. | Organization |
| Feedback | US-13 | As a user, I want to submit post-project feedback so that both parties have a credibility record. | Both |
| Admin Moderation | US-14 | As an admin, I want to moderate platform content so that the marketplace remains trustworthy. | Admin |
| System Logging | US-15 | As an admin, I want to review system logs so that I can monitor platform health. | Admin |
| Notifications | US-16 | As a user, I want to receive in-platform notifications for key events so that I stay informed. | All |
| Messaging | US-17 | As a project participant, I want to exchange messages with my counterpart so that we can coordinate. | Both |
| Analytics | US-18 | As a user, I want to see productivity analytics on my dashboard so that I can track my progress. | Both |
| Profile Enhancements | US-19 | As a student, I want my profile to display verified achievements and portfolio links so that I appear credible to organizations. | Student |
| Smart Matching | US-20 | As a student, I want personalized project recommendations ranked by my skills and profile so that I can find the best matches quickly. | Student |

## 2.3 User-Action Matrix

The following table maps each user role to the actions they are authorized to perform on the platform.

| Action | Student | Organization | Admin |
|---|---|---|---|
| Register / Login | Yes | Yes | Yes |
| Create/edit student profile | Yes | No | No |
| Create/edit organization profile | No | Yes | No |
| Post a project listing | No | Yes | No |
| Browse and search projects | Yes | No | No |
| View smart recommendations | Yes | No | No |
| Apply to a project | Yes | No | No |
| Accept/reject applications | No | Yes | No |
| Submit a deliverable | Yes (if accepted) | No | No |

| Action | Student | Organization | Admin |
|---|---|---|---|
| Review and approve deliverables | No | Yes | No |
| Mark project complete | No | Yes | No |
| Submit feedback | Yes (if project complete) | Yes (if project complete) | No |
| Send/receive project messages | Yes (if accepted) | Yes (if project owner) | No |
| View notifications | Yes | Yes | Yes |
| View analytics dashboard | Yes (student metrics) | Yes (org metrics) | No |
| View system logs | No | No | Yes |
| Moderate content | No | No | Yes |

## 2.4 Constraints

- **Technology Stack:** Python 3.11 (backend), Node.js 18+ (frontend), PostgreSQL 15 (database), Docker for containerization.
- **Authentication:** JWT tokens only. No session cookies or OAuth social login.
- **Schema Management:** Base.metadata.create_all() used for initial table creation. ALTER TABLE required for column additions (no Alembic migration runner in current version).
- **Frontend-Backend Coupling:** The React frontend communicates exclusively via the REST API. No direct database access from the client.
- **File Uploads:** Deliverable submissions are text/URL only in the current MVP. Binary file hosting is not supported.
- **Single Developer:** The system is developed and maintained by a single developer, constraining availability of parallel feature development.

## 2.5 Assumptions

- Users have access to a modern web browser (Chrome, Firefox, or Edge) and a stable internet connection.
- Organizations are assumed to be legitimate entities. Account verification and KYC are out of scope for MVP.
- Student profile data (university, major, skills) is self-reported and not verified against institutional records in this version.
- Deliverable content is trusted by organizations at time of review. Automated plagiarism or quality checking is out of scope.
- The recommendation algorithm assumes student skill and bio fields are populated; users with incomplete profiles will receive lower match scores.

# 3. Functional Requirements

The following table enumerates all functional requirements for the MicroMatch MVP. Each requirement is mapped to its corresponding User Story (US) and Use Case (UC), and includes acceptance criteria and implementation notes.

## 3.1 Authentication and Account Management

| ID | US/UC | Requirement | Acceptance Criteria | Notes |
|---|---|---|---|---|
| FR-1 | US-1, UC-1 | The system shall allow any visitor to register a new account by providing name, email, password, and role (student, organization, or admin). | Account is created; user receives a JWT token on successful registration; duplicate email returns 400. | Passwords hashed with bcrypt. |
| FR-2 | US-1, UC-1 | The system shall support Role-Based Access Control (RBAC) enforced at every API endpoint. Students, organizations, and admins may only invoke endpoints permitted for their role. | Calling an endpoint with the wrong role returns HTTP 403 Forbidden. Unauthenticated requests return HTTP 401. | Implemented via FastAPI Depends() injection. |

## 3.2 Profile Management

| ID | US/UC | Requirement | Acceptance Criteria | Notes |
|---|---|---|---|---|
| FR-3 | US-2, UC-2 | The system shall allow students to create and edit their profile, including university, major, graduation year, skills (comma-separated), bio, and portfolio links. | Profile is saved and retrievable via GET /student/profile; all fields visible in view mode; skills render as chips. | portfolio_links and badges stored as comma-separated text. |
| FR-4 | US-3, UC-3 | The system shall allow organizations to create and edit their profile, including organization name, industry, website, and description. | Profile is saved and retrievable; visible to students viewing project details. | |
| FR-19 | US-19, UC-2 | The system shall automatically compute and award student achievement badges based on verified activity milestones. Students shall | Badges are updated on project completion and feedback submission events. PUT /student/profile/enhance does not accept a badges field. | 5 badge definitions: first_project, three_projects, ten_projects, top_rated, rising_star. |

| ID | US/UC | Requirement | Acceptance Criteria | Notes |
|---|---|---|---|---|
| | | not be able to manually set badge values. | | |

## 3.3 Project Management

| ID | US/UC | Requirement | Acceptance Criteria | Notes |
|---|---|---|---|---|
| FR-5 | US-4, UC-4 | The system shall allow organizations to post project listings with title, description, required skills, and duration. | Project appears in GET /projects with status=open; visible to all authenticated users. | |
| FR-6 | US-5, UC-5 | The system shall allow students to browse and search all open project listings. Search shall filter by keyword matching title and required skills. | GET /projects returns all open projects; keyword filter narrows results. | |

## 3.4 Application Workflow

| ID | US/UC | Requirement | Acceptance Criteria | Notes |
|---|---|---|---|---|
| FR-7 | US-6, UC-6 | The system shall allow students to apply to any open project. A student may not submit duplicate applications to the same project. | Application is created with status=pending; UNIQUE constraint on (student_id, project_id); duplicate returns 400. | |
| FR-8 | US-8, UC-7 | The system shall allow students to retrieve all of their applications with their current status. | GET /applications/me returns all applications for the authenticated student with status field. | |
| FR-9 | US-9, UC-7 | The system shall allow organizations to accept or reject applications for their own projects. Only the owning organization may change application status. | PATCH /applications/{id}/status updates status; non-owning org returns 403; student notified on status change. | |

## 3.5 Deliverables and Project Completion

| ID | US/UC | Requirement | Acceptance Criteria | Notes |
|---|---|---|---|---|
| FR-10 | US-10, UC-8 | The system shall allow accepted students to submit a project deliverable (text, | POST /applications/{id}/deliverables creates deliverable; only accepted student may | |

| ID | US/UC | Requirement | Acceptance Criteria | Notes |
|---|---|---|---|---|
| | | link, or file reference) for their accepted application. | submit; duplicate submission returns 400. | |
| FR-11 | US-11, UC-8 | The system shall allow organizations to review submitted deliverables and either approve them or request a revision. | Organization can update deliverable status to accepted or rejected; student notified. | |
| FR-12 | US-12, UC-9 | The system shall allow organizations to mark a project complete. Marking complete triggers badge computation for the accepted student. | Project status changes to completed; award_badges() called within the same transaction; student notified. | |

## 3.6 Feedback System

| ID | US/UC | Requirement | Acceptance Criteria | Notes |
|---|---|---|---|---|
| FR-13 | US-13, UC-10 | The system shall allow both students and organizations to submit a 1-5 star rating and optional comment for a project after it is marked complete. Each user may submit one feedback entry per project. | POST /feedback creates feedback; rating must be 1-5; UNIQUE(user_id, project_id) prevents duplicates; rating feeds into student average displayed on profile. | |

## 3.7 Notifications

| ID | US/UC | Requirement | Acceptance Criteria | Notes |
|---|---|---|---|---|
| FR-16 | US-16, UC-14 | The system shall generate in-platform notifications for the following events: new application received, application accepted, application rejected, deliverable submitted, deliverable approved, project marked complete, and new message sent. | Notification row inserted within the same DB transaction as the triggering event; GET /notifications returns unread count; PUT /notifications/{id}/read marks as read. | Notifications do not commit independently; atomicity is caller-owned. |

## 3.8 Messaging

| ID | US/UC | Requirement | Acceptance Criteria | Notes |
|---|---|---|---|---|
| FR-17 | US-17, UC-12 | The system shall allow the accepted student and the owning organization for a project to exchange messages scoped to that project. All other users shall be denied access. | POST /projects/{id}/messages creates message; GET /projects/{id}/messages returns history oldest-to-newest; non-participant returns 403; message triggers notification to other participant. | require_project_participant dependency enforces access. |

## 3.9 Analytics

| ID | US/UC | Requirement | Acceptance Criteria | Notes |
|---|---|---|---|---|
| FR-18 | US-18, UC-13 | The system shall provide role-specific analytics. Students see: applications submitted, accepted applications, completed projects, average feedback rating. Organizations see: total projects posted, active projects, completed projects, total unique applicants. | GET /analytics/student returns four metrics for authenticated student; GET /analytics/organization returns four metrics for authenticated organization; cross-role access returns 403. | Computed via SQL aggregates (func.count, func.avg, func.distinct) per request. |

## 3.10 Smart Matching and Recommendations

| ID | US/UC | Requirement | Acceptance Criteria | Notes |
|---|---|---|---|---|
| SA-1 | US-20, UC-11 | The system shall provide a personalized ranked list of open projects for authenticated students, scored by a weighted five-factor algorithm. Results shall be deterministic for the same inputs. | GET /recommendations?top_n=N returns N projects with score and breakdown; scores are reproducible; endpoint blocked for non-students. | See Section 5 for full algorithm specification. |

## 3.11 Administration and Logging

| ID | US/UC | Requirement | Acceptance Criteria | Notes |
|---|---|---|---|---|
| FR-14 | US-14, UC-15 | The system shall provide administrators with tools to view and moderate platform content. | Admin-only GET /admin/logs endpoint; non-admin returns 403. | |

| ID | US/UC | Requirement | Acceptance Criteria | Notes |
|---|---|---|---|---|
| FR-15 | US-15, UC-15 | The system shall log every HTTP request with user identity, role, endpoint, method, status code, and timestamp via a cross-cutting middleware. | Every request produces a SystemLog row; unauthenticated requests log user_id=null; no request is silently skipped. | BaseHTTPMiddleware intercepts all requests regardless of endpoint. |

## 3.12 Functional Requirements Traceability Matrix

| FR ID | User Story | Use Case | Feature Area | Sprint Delivered |
|---|---|---|---|---|
| FR-1 | US-1 | UC-1 | Authentication | Sprint 1 |
| FR-2 | US-1 | UC-1 | RBAC | Sprint 1 |
| FR-3 | US-2 | UC-2 | Student Profile | Sprint 1 |
| FR-4 | US-3 | UC-3 | Org Profile | Sprint 1 |
| FR-5 | US-4 | UC-4 | Project Posting | Sprint 1 |
| FR-6 | US-5 | UC-5 | Project Discovery | Sprint 1 |
| FR-7 | US-6 | UC-6 | Applications | Sprint 2 |
| FR-8 | US-8 | UC-7 | Application Tracking | Sprint 2 |
| FR-9 | US-9 | UC-7 | Accept/Reject | Sprint 2 |
| FR-10 | US-10 | UC-8 | Deliverables | Sprint 3 |
| FR-11 | US-11 | UC-8 | Deliverable Review | Sprint 3 |
| FR-12 | US-12 | UC-9 | Project Completion | Sprint 3 |
| FR-13 | US-13 | UC-10 | Feedback | Sprint 3 |
| FR-14 | US-14 | UC-15 | Admin Moderation | Sprint 3 |
| FR-15 | US-15 | UC-15 | System Logging | Sprint 3 |
| FR-16 | US-16 | UC-14 | Notifications | Sprint 4 Wk 1 |
| FR-17 | US-17 | UC-12 | Messaging | Sprint 4 Wk 3 |
| FR-18 | US-18 | UC-13 | Analytics | Sprint 4 Wk 3 |
| FR-19 | US-19 | UC-2 | Profile Enhancements | Sprint 4 Wk 3 |
| SA-1 | US-20 | UC-11 | Smart Matching | Sprint 4 Wk 2 |

# 4. Non-Functional Requirements

Non-functional requirements define the quality attributes of MicroMatch that govern how the system behaves, not what it does. Each NFR below includes a rationale explaining why it is essential to the business goals of the platform.

**NFR-1 — Security**

- **Requirement:** All API endpoints shall require JWT authentication. Endpoints with role restrictions shall return HTTP 403 for unauthorized roles. Project messaging endpoints shall additionally verify that the requestor is an accepted participant in the specific project.
- **Business Rationale:** MicroMatch handles sensitive personal and professional data including application decisions, communications, and feedback ratings. A single authorization failure could expose private student or organization data, undermine trust in the platform, and violate user privacy expectations. Security is the highest-priority NFR and directly enables the business goal of building trust between parties.
- **Measurability:** 100% of protected endpoints return 401 for unauthenticated requests and 403 for role violations. Verified by 251 automated tests.

**NFR-2 — Correctness**

- **Requirement:** All computed metrics (badges, analytics, ratings) shall be derived from live database records at query time. No pre-computed or cached metric values shall be served to users.
- **Business Rationale:** The platform's credibility system depends on accurate metrics. A student's badge count must reflect actual completed projects; a misleading analytics value could misrepresent a student's record to organizations. Correctness enables the business goal of trust and credibility. Stale metrics would undermine the platform's value proposition as a verification tool.
- **Measurability:** Badge values recomputed transactionally on each triggering event. Analytics queries use SQL aggregates per request with no intermediate cache layer.

**NFR-3 — Testability**

- **Requirement:** All backend business logic shall be unit-testable without a running PostgreSQL instance. Test coverage shall meet or exceed 90% for all new core business logic modules.
- **Business Rationale:** As a single-developer project, automated tests are the primary regression safety net. Without high testability, changes to one module can silently break another. The 90% coverage threshold was selected to ensure that all primary business flows (application lifecycle, badge awards, RBAC enforcement) are verified on every commit. This directly supports the engineering goal of maintainable, production-ready software.
- **Measurability:** 251 automated tests pass against SQLite in-memory DB. Coverage >= 96% measured via pytest-cov.

**NFR-4 — Performance**

- **Requirement:** Analytics and recommendation queries shall complete in a single database round-trip using SQL aggregate functions. The recommendation engine shall return a ranked result set in under 500ms for up to 1,000 open projects.
- **Business Rationale:** Students and organizations interact with the analytics dashboard and recommendations feed on the home page — the first thing they see after login. Slow queries create a poor first impression and reduce engagement. The single-round-trip constraint prevents N+1 query patterns that would degrade proportionally as the project catalog grows.
- **Measurability:** Analytics: single func.count/func.avg query per endpoint. Recommendations: one SELECT on projects + in-memory scoring. No ORM lazy loading on critical paths.

**NFR-5 — Maintainability**

- **Requirement:** Business logic shall not leak into Pydantic schema files or SQLAlchemy model files. All scoring logic for the matching engine shall be isolated in a single pure function module (matching_engine.py) that can be modified without changing any router file.
- **Business Rationale:** MicroMatch is a growing platform. The matching algorithm scoring weights will require tuning as real usage data becomes available. If scoring logic is scattered across router files, changes become error-prone and risky. Strict module layering reduces the change surface for any given modification, enabling faster iteration and lower defect rates.
- **Measurability:** matching_engine.py has zero imports from app.routers. BADGE_DEFINITIONS list — adding a badge requires one entry, zero router changes.

**NFR-6 — Observability**

- **Requirement:** Every HTTP request to the system shall produce a persisted SystemLog record containing: timestamp, authenticated user ID (null for unauthenticated), user role, endpoint path, HTTP method, and response status code.
- **Business Rationale:** Platform operators need full visibility into system behavior to diagnose issues and monitor for abuse. Without request-level logging, debugging production problems requires guesswork. The logging middleware enables the business goal of platform integrity by giving administrators a complete audit trail of all actions taken on the system.
- **Measurability:** 100% of requests produce a SystemLog row, verified by dedicated middleware tests. Admin endpoint confirms log retrieval.

**NFR-7 — Deployability**

- **Requirement:** The backend and database shall be deployable in an isolated environment using a single docker compose up command. The frontend shall be deployable independently as a static build.
- **Business Rationale:** Reproducible deployment is essential for onboarding new contributors and submitting the project for evaluation. A system that requires complex manual setup steps loses portability and increases the risk of environment-specific defects. Docker Compose encapsulates the database dependency cleanly, enabling anyone with Docker installed to run the full stack in minutes.
- **Measurability:** docker compose up -d starts PostgreSQL. uvicorn app.main:app starts API. npm start starts frontend. No manual SQL scripts required for initial table creation.

**NFR-8 — Usability**

- **Requirement:** All asynchronous UI components shall display a loading state while data is being fetched and an error state if the fetch fails. Empty states shall be shown when a list or collection contains no items, rather than rendering a blank area.
- **Business Rationale:** Users who see blank screens or spinners with no feedback lose confidence in the platform and abandon workflows. The marketplace value of MicroMatch depends on students and organizations completing the full application lifecycle. Clear loading, error, and empty states reduce friction at every step and reinforce the platform's reliability.
- **Measurability:** RecommendationWidget, AnalyticsDashboard, MessagesHubPage, and MyApplications all implement loading skeletons, error banners, and empty-state illustrations.

# 5. Algorithmic Component — Smart Matching and Ranking

## 5.1 Problem Statement

A core limitation of traditional job boards is that they rely exclusively on keyword search. A student searching for "Python" receives every Python-tagged project regardless of whether the project is appropriate for their graduation year, whether they have demonstrated a track record of completing projects, or whether the project description aligns with their stated interests. This creates signal noise that forces users to manually filter through irrelevant results.

MicroMatch addresses this by replacing keyword-only search with a personalized ranked recommendation feed. The algorithm must solve the following problem: given a student's profile (skills, major, bio, graduation year, activity history, completion record) and a set of open project listings, produce a ranked list of projects ordered by predicted relevance to that specific student, with the ranking deterministic and auditable.

## 5.2 Alignment with System Scope and User Stories

The Smart Matching Algorithm directly fulfills:

- **US-20 / SA-1** — As a student, I want personalized project recommendations ranked by my skills and profile so that I can find the best matches quickly.
- **FR-6 (extended)** — The system shall allow students to discover open projects. The algorithm provides an enhanced discovery layer beyond keyword search.
- **NFR-3 (Testability)** — The algorithm is a pure function, enabling ≥90% coverage without database dependencies.
- **NFR-4 (Performance)** — The algorithm operates in-memory on a pre-fetched project list, keeping the GET /recommendations response under 500ms.
- **NFR-5 (Maintainability)** — All scoring weights are isolated in matching_engine.py and modifiable without touching any router file.

The recommendation system is MicroMatch's Significant Algorithm (SA-1) as designated in the Sprint 4 backlog. It represents the platform's primary differentiator from a simple job board.

## 5.3 Algorithm Logic and Scoring Formula

The algorithm is implemented as a set of pure Python functions in app/services/matching_engine.py. It accepts two inputs: a StudentDTO and a list of ProjectDTOs (plain Python dataclasses, fully decoupled from SQLAlchemy). It returns a list of ScoredProject objects sorted by score descending.

Each project is evaluated against five independent scoring factors. The final score is a weighted linear combination:

```
score(p, s) = (0.40 × skill_score) + (0.20 × experience_score) + (0.15 ×
   interest_score) + (0.10 × activity_score) + (0.15 × success_score)
```

All component scores are in the range [0, 100]. The final match_score is therefore also bounded [0, 100], where 100 represents a perfect match on all five factors.

| Factor | Weight | Formula / Logic | Rationale |
|---|---|---|---|
| skill_score | 40% | Intersection over required skills: \|student_skills ∩ project_required_skills\| / \|project_required_skills\|. Skills tokenized from comma-separated strings, lowercased, compared as sets. If project has no required skills → 50.0 (neutral). If student has no skills → 0.0. | Skills are the primary matching criterion. Normalised by required skill count rather than union (Jaccard), measuring how well a student covers the project's specific needs. |
| experience_score | 20% | Tiered base from completed_project_count: 0→30, 1→55, 2→70, 3–5→80, 6+→90. Adjusted by graduation year proximity: graduating in 1 yr +5, 2 yrs +2, 3+ yrs -5. Duration penalty applied for inexperienced students on long projects: 0 completed + >12 wks → -15; <=1 completed + >8 wks →-5 | Matches project complexity to student experience level. Avoids recommending long, demanding projects to students with no prior completions. Graduation proximity serves as a proxy for academic stage. |
| interest_score | 15% | Weighted keyword blend: major tokens vs. project keywords (60% weight) + bio keywords vs. project keywords using Jaccard similarity (40% weight). Stop words filtered; case-insensitive. If project has no description → 50.0 (neutral). | Captures domain alignment beyond formal skill tags. A student whose bio mentions "machine learning" or "web development" is implicitly expressing interest in those domains, complementing the skill match. |
| activity_score | 10% | Tiered score based on applications submitted in the last 30 days: 0→10, 1→40, 2→65, 3→80, 4+→100. Application frequency is used as the activity signal since no last_login field exists in the schema. | Rewards recently active students who are actively seeking opportunities. Higher activity indicates current engagement with the platform and readiness to take on a project. |
| success_score | 15% | Weighted blend of three historical dimensions: (1) completion ratio = completed / accepted × 100 (wt. 40%), (2) delivery rate = accepted_deliverables / completed × 100 (wt. 30%), (3) normalised feedback rating = (avg_rating − 1) / 4 × 100 (wt. 30%). New students with no history → 50.0 (neutral). All denominators guarded against divide-by-zero. | Reflects the student's track record of following through. Rewards students who complete projects, submit quality deliverables, and earn positive feedback. New students are not penalised — they receive a neutral score that neither boosts nor suppresses their ranking. |

## 5.4 Algorithmic Properties

| Property | Description |
|---|---|
| Determinism | The scoring functions are pure functions with no side effects or random number generation. The graduation year comparison resolves current_year once at call |

| | |
|---|---|
| | time via datetime.now (timezone.utc) year. Given the same inputs, rank_projects() always produces identical ranked output. |
| Time Complexity | O(N × (S + R)) for scoring — one pass over N projects, each requiring O(S + R) set operations where S = student skill count, R = required skill count for that project. O(N log N) for sort. Total: O(N × (S + R) + N log N). |
| Space Complexity | O(N) for the scored list held in memory during sorting. |
| Score Range | Each factor is bounded [0, 100]. The weighted sum is therefore bounded [0, 100]. A score of 100.0 represents a perfect match on all five factors |
| Graceful Degradation | Empty bio → interest_score bio component = 0.0. Empty skills → skill_score = 0.0. New student with no history → success_score = 50.0 (neutral). Remaining factors still contribute, preventing null results for incomplete profiles. |
| Divide-by-Zero Safety | All denominators explicitly guarded throughout the engine. parse_skills() handles None, empty string, and extra whitespace safely. |

## 5.5 API Integration and Data Flow

| Step | Action | Component |
|---|---|---|
| 1 | Student calls GET /recommendations?top_n=10 | React SPA → FastAPI Router |
| 2 | require_role('student') dependency validates JWT and role | app/core/dependencies.py |
| 3 | Router fetches student's StudentProfile from PostgreSQL | SQLAlchemy ORM → PostgreSQL |
| 4 | Router queries DB for completed count, accepted count, accepted deliverables, avg rating, and recent application count; assembles StudentDTO via _build_student_dto() | app/routers/recommendations.py |
| 5 | Router fetches all Project rows with status='open' and maps to ProjectDTOs via _build_project_dto() | SQLAlchemy ORM → PostgreSQL |
| 6 | Router calls rank_projects(profile_dict, project_list) — no DB calls inside | app/services/matching_engine.py |
| 7 | Engine computes all five scores per project using pure functions, returns sorted ScoredProject list[:top_n] | Pure Python in-memory |
| 8 | Router assembles RecommendationResponse DTOs with project + score + breakdown | app/routers/recommendations.py |
| 9 | Response serialized by Pydantic and returned to client | FastAPI response model |

## 5.6 Test Coverage

The matching engine is covered by dedicated unit tests in backend/tests/test_matching_engine.py and endpoint tests in backend/tests/test_recommendations.py. Key test scenarios include:

- Perfect skill overlap returns skill_score = 100.0; zero overlap returns 0.0; no required skills returns 50.0 (neutral).
- Experience score tiers verified: 0 completed → base 30, 1 → 55, 2 → 70, 3–5 → 80, 6+ → 90, with graduation year adjustments and duration penalties applied correctly.
- Interest score: matching major tokens boost score; empty bio yields bio_score = 0.0; Jaccard logic verified for bio overlap.
- Activity score tiers verified: 0 apps → 10, 1 → 40, 2 → 65, 3 → 80, 4+ → 100.
- Success score: new student with zero history returns 50.0 (neutral); completion ratio, delivery rate, and feedback rating blend verified for known inputs.
- Determinism test: two calls with identical StudentDTO and ProjectDTO inputs produce identical ranked output.
- top_n=0 rejected at API layer (ge=1 Pydantic validator) before reaching engine; top_n slicing verified.
- Student with empty profile (no skills, no bio) still receives scored results from experience, activity, and success factors.
- Endpoint tests verify 403 for non-student roles, 404 when no student profile exists, and empty list when no open projects are available.